

MASTER PROMPT — COMPLETE WEB SURVIVAL ROGUELITE GAME

Project Working Title

ORBITAL SURVIVOR

0. YOUR ROLE

Act as a complete senior game development team consisting of:

- Lead Game Designer
- Senior TypeScript Engineer
- Senior Phaser Engineer
- Gameplay Programmer
- Combat Systems Designer
- Roguelite Systems Designer
- Game Economy Designer
- Mobile Web Optimization Engineer
- UI/UX Designer
- Technical Artist
- VFX Designer
- Audio Systems Designer
- QA Engineer
- Software Architect
- Performance Engineer
- DevOps / Web Deployment Engineer

You are responsible for designing and implementing a complete production-quality browser game.

Do not only provide conceptual explanations.

Create the actual project architecture, source code, configuration files, game systems, data structures, testing strategy, and deployment configuration.

The project must remain runnable throughout development.

Never leave the project in a knowingly broken state.

1. PROJECT OBJECTIVE

Create an original 2D survival roguelite action game for web browsers.

The core gameplay concept is:

The player manually moves a character.

Weapons attack automatically.

Large numbers of enemies continuously approach the player.

Enemies drop experience resources.

The player collects experience.

When enough experience is collected, the player levels up.

The game pauses.

Three upgrades are presented.

The player selects one.

Combat resumes.

Weapons and passive modules become increasingly powerful.

Some weapon/passive combinations unlock evolved weapons.

Enemy density and difficulty progressively increase.

Elite enemies appear.

Boss enemies appear.

The player must survive until the end of the stage.

Initial stage duration:

10 minutes.

2. ORIGINALITY REQUIREMENT

The game may use general survival-roguelite genre principles.

However, this must be an original game.

DO NOT copy any copyrighted or proprietary assets from existing games.

Do not copy:

- Survivor.io characters
- Survivor.io enemy designs
- Survivor.io weapon names
- Survivor.io interface
- Survivor.io map designs
- Survivor.io icons
- Survivor.io sounds
- Survivor.io story
- Survivor.io progression values
- Survivor.io artwork
- Vampire Survivors artwork
- Other commercial game assets

Create an original identity.

3. TARGET PLATFORM

Primary platform:

Mobile Web App

Supported environments:

- iPhone Safari
- iPad Safari
- Android Chrome
- Desktop Chrome
- Desktop Edge
- Desktop Firefox
- macOS Safari

Game orientation:

Portrait-first.

Desktop must also be supported.

Recommended logical game viewport:

720 × 1280

Scale responsively to device screen.

Support:

- touch input


```

ProjectileManager.ts | | |—— TargetingSystem.ts | | |—— CollisionManager.ts | | |——
StatusEffectSystem.ts | | |—— weapons/ | | |—— WeaponManager.ts | | |——
BaseWeapon.ts | | |—— ProjectileWeapon.ts | | |—— OrbitWeapon.ts | | |——
AreaWeapon.ts | | |—— ChainWeapon.ts | | |—— MissileWeapon.ts | | |—— evolved/ | | |
|—— progression/ | | |—— ExperienceSystem.ts | | |—— LevelSystem.ts | | |——
UpgradeSystem.ts | | |—— EvolutionSystem.ts | | |—— MetaProgressionSystem.ts | | |
|—— pickups/ | | |—— Pickup.ts | | |—— ExpGem.ts | | |—— GoldPickup.ts | | |——
HealthPickup.ts | | |—— MagnetPickup.ts | | |—— bosses/ | | |—— Boss.ts | | |——
BossAI.ts | | |—— GuardianBoss.ts | | |—— ui/ | | |—— HUD.ts | | |——
VirtualJoystick.ts | | |—— HealthBars.ts | | |—— ExperienceBars.ts | | |—— UpgradePanel.ts |
| |—— WeaponSlotUI.ts | | |—— DamageNumberManager.ts | | |—— MobileSafeArea.ts | |
| |—— data/ | | |—— enemies.ts | | |—— weapons.ts | | |—— passives.ts | | |——
evolutions.ts | | |—— stages.ts | | |—— characters.ts | | |—— progression.ts | | |——
save/ | | |—— SaveManager.ts | | |—— SaveData.ts | | |—— SaveMigration.ts | | |——
audio/ | | |—— AudioManager.ts | | |—— debug/ | | |—— DebugOverlay.ts | | |——
DeveloperConsole.ts | | |—— utils/ | | |—— index.html |—— package.json |—— tsconfig.json
|—— vite.config.ts |—— README.md

```

Keep modules focused.

Avoid massive single files.

6. GAME WORLD

Use an effectively large scrolling battlefield.

The player should remain near the visual center.

The world should feel much larger than the viewport.

Use camera-follow behavior.

Initial map theme:

Abandoned high-tech colony.

Create an abstract procedural floor using simple tiles or shapes if real artwork is unavailable.

The project MUST run without external proprietary assets.

Use placeholder graphics generated with Phaser primitives when required.

7. PLAYER

Initial playable character:

Aegis-01

Concept:

Compact autonomous exploration/combat robot.

Initial statistics:

HP = 100 Move Speed = configurable Armor = 0 Critical Chance = 5% Critical Damage = 150% Pickup
Radius = configurable Health Regeneration = 0

Player movement must support:

- virtual joystick
- WASD
- arrow keys

Normalize diagonal speed.

Do not allow diagonal movement to be faster than horizontal movement.

8. MOBILE VIRTUAL JOYSTICK

Create a high-quality touch joystick.

Requirements:

- left-side control area
- base circle
- movable thumb
- adjustable dead zone
- analog output
- normalized Vector2
- auto-centering
- multi-touch safe
- does not block UI buttons
- responsive positioning
- safe-area aware

Desktop:

WASD and arrow keys.

Optional:

Mouse click/drag movement mode may be added later.

9. PLAYER STATS ARCHITECTURE

Create modular player stats.

Required stats:

maxHealth moveSpeed armor damageMultiplier attackSpeedMultiplier cooldownReduction
criticalChance criticalDamage pickupRadius healthRegeneration projectileSpeedMultiplier
projectileSizeMultiplier areaMultiplier durationMultiplier experienceMultiplier

Support modifiers.

Modifier types:

Flat AdditivePercent MultiplicativePercent

Do not directly modify arbitrary values across many scripts.

PlayerStats should be the authoritative stat source.

10. DAMAGE SYSTEM

Implement generic damage architecture.

Create:

DamageInfo

Fields:

amount damageType source critical knockback hitPosition

Initial damage types:

Physical Energy Explosive

Enemy and player health components should receive damage through the same general damage interface when practical.

Support:

- critical hits
- armor
- invulnerability frames
- knockback
- death event

- hit event
-

11. ENEMY ROSTER

Create at least six standard enemy categories.

E01 Crawler

Basic enemy.

Behavior:

Move directly toward player.

Stats:

Low HP Normal speed Melee damage

E02 Scout Drone

Fast enemy.

Behavior:

Rapid pursuit.

Stats:

Very low HP High speed

E03 Heavy Drone

Tank enemy.

Stats:

High HP Slow speed High contact damage

E04 Spitter

Ranged enemy.

Behavior:

Maintain approximate distance.

Periodically fire projectile.

E05 Charger

Behavior:

Approach normally.

Pause.

Telegraph.

Charge rapidly at player.

Cooldown.

E06 Swarm Unit

Very weak enemy.

Spawn in large groups.

Used to create density.

12. ELITE ENEMY

Create:

Elite Sentinel

Features:

- larger visual size
- increased HP
- increased damage
- recognizable appearance
- special attack
- better rewards

At least one special behavior:

Radial projectile attack.

Elite should appear around:

03:00 07:00

Configurable from stage data.

13. BOSS

Create one complete boss:

Guardian Prime

Encounter:

At 10:00.

Boss must have at least three attacks.

Attack 1: Radial projectile burst.

Attack 2: Charge attack.

Attack 3: Area warning zones followed by explosions.

Boss must telegraph dangerous attacks.

Boss phases:

Phase 1: 100–60% HP

Phase 2: 60–30%

Phase 3: Below 30%

Later phases increase attack frequency.

Boss must have:

Boss HP bar Boss name Attack cooldown controller Phase state Death sequence Reward event

14. ENEMY SPAWN DIRECTOR

Implement a centralized SpawnDirector.

Do not let individual enemy classes control global spawning.

SpawnDirector inputs:

elapsedTime currentStage difficultyMultiplier activeEnemyCount

Responsibilities:

- select enemy type
- determine spawn count
- determine spawn frequency
- determine spawn location
- activate elite events
- activate boss event
- enforce maximum active enemies

Enemy spawn position should usually be outside visible camera bounds.

Use a configurable spawn margin.

15. STAGE TIMELINE

Create data-driven stage events.

Initial stage:

STAGE 01 — Abandoned Colony

Example timeline:

00:00 Crawler

00:30 Crawler + Scout Drone

01:30 Increase spawn density

02:00 Introduce Heavy Drone

03:00 Elite Sentinel

04:00 Introduce Spitter

05:00 High-density Swarm event

06:00 Introduce Charger

07:00 Elite Sentinel

08:00 Extreme wave

09:00 Final preparation phase

10:00 Stop normal waves Spawn Guardian Prime

Do not hard-code this directly into GameScene.

Create stage configuration data.

16. DIFFICULTY SCALING

Create a configurable difficulty model.

Variables:

Enemy HP multiplier Enemy damage multiplier Enemy speed multiplier Spawn rate multiplier Maximum active enemies Elite multiplier

Difficulty increases according to elapsed time.

Do not create abrupt unfair jumps except for deliberate wave events.

Use tunable interpolation or curves.

17. OBJECT POOLING

Mandatory.

Implement object pools for:

- enemies
- projectiles
- enemy projectiles
- EXP gems
- explosions
- damage numbers
- temporary effects
- missiles

Avoid frequent JavaScript object allocation in hot gameplay loops.

Avoid repeatedly creating and destroying Phaser GameObjects.

Example initial pools:

Enemies: 300

Projectiles: 250

EXP gems: 500

Enemy projectiles: 150

VFX: 100

Pools may expand if necessary but should reuse objects.

18. PERFORMANCE TARGETS

Desktop target:

60 FPS 500–1000 lightweight enemies if hardware permits

Modern phone target:

60 FPS 300–600 enemies

Fallback:

30 FPS

Create PerformanceManager.

Monitor:

FPS activeEnemies activeProjectiles activePickups

Optional adaptive behavior:

If average FPS falls below threshold:

- reduce damage number frequency
- reduce particles
- reduce decorative effects
- merge EXP pickups
- reduce maximum enemy count slightly

Gameplay logic must remain consistent.

19. SPATIAL PARTITIONING

Do not perform naive global nearest-enemy searches every frame for every weapon.

Implement:

SpatialGrid

Each enemy registers its position.

Grid cells contain enemy references.

Support queries:

getEnemiesInRadius()

getNearestEnemy()

getNearestEnemies()

getRandomEnemyInRadius()

Update spatial cells efficiently.

Use it for:

- auto targeting
- AoE damage
- chain weapons
- enemy cluster detection

20. WEAPON SYSTEM

Create data-driven weapons.

The player can initially hold:

Maximum 6 weapons.

Each base weapon:

Level 1–5.

Weapons automatically attack.

Weapon system must separate:

definition data runtime state attack behavior

21. WEAPON 1 — PULSE BLASTER

Type:

Auto-target projectile.

Behavior:

Find nearest enemy.

Fire energy projectile.

Levels:

Lv1: Basic projectile

Lv2: Increase damage

Lv3: +1 projectile

Lv4: Increase fire rate

Lv5: Increase penetration and damage

Primary parameters:

damage cooldown projectileCount projectileSpeed pierce

22. WEAPON 2 — ORBIT DRONES

Objects rotate around player.

Enemies touching drone receive damage.

Levels improve:

drone count damage rotation speed size orbit radius

Avoid damage every rendering frame.

Use controlled damage tick interval.

23. WEAPON 3 — PLASMA FIELD

Continuous circular damage field around player.

Apply damage using fixed tick interval.

Levels improve:

radius damage tick rate

Use spatial grid radius query.

24. WEAPON 4 — RICOCHET DISC

Launch disc toward enemy.

After hit:

find next nearby valid target.

Bounce between enemies.

Prevent repeatedly hitting same enemy within a single bounce chain where appropriate.

Upgrade:

damage bounce count disc count cooldown

25. WEAPON 5 — ARC NODE

Chain energy weapon.

Initial target:

nearest valid enemy.

Then jump between nearby targets.

Visualize temporary electric arcs using Phaser graphics or pooled line effects.

Upgrade:

damage chain count range cooldown

26. WEAPON 6 — MICRO MISSILE POD

Select enemy cluster.

Launch missile.

Missile travels toward target area.

Explosion damages enemies in radius.

Upgrade:

damage missile count explosion radius cooldown

Use spatial grid for AoE.

27. PASSIVE MODULES

Player can hold maximum:

6 passives.

Implement:

P01 Power Amplifier Damage increase.

P02 Cooling Module Cooldown reduction.

P03 Mobility Servo Movement speed increase.

P04 Armor Plating Max HP / armor increase.

P05 Magnetic Collector Pickup radius increase.

P06 Energy Capacitor Area/duration improvement.

Each:

Level 1–5.

All values should come from data files.

28. EVOLUTION SYSTEM

Create data-driven weapon evolutions.

Examples:

Pulse Blaster Lv5

+ Power Amplifier = Twin Pulse Array

Orbit Drones Lv5

+ Cooling Module = Quantum Orbit

Plasma Field Lv5

+ Energy Capacitor = Plasma Reactor

Ricochet Disc Lv5

+ Mobility Servo = Hyper Disc

Arc Node Lv5

+ Magnetic Collector = Storm Network

Micro Missile Pod Lv5

+ Armor Plating = Siege Missile Array

Evolution recipes must be data-defined.

Do not implement recipes with hardcoded nested if-statements.

29. EXPERIENCE SYSTEM

Enemies drop Energy Cores.

Three values:

Small Medium Large

Energy Cores remain on battlefield until collected unless optimization merges them.

When within pickup radius:

Accelerate toward player.

On collision:

Add experience.

Return pickup to pool.

30. EXP OPTIMIZATION

Large quantities of EXP objects may become expensive.

Implement an optimization strategy.

Possible approach:

If too many gems exist:

merge nearby gems.

Store accumulated EXP value.

Replace multiple small gems with fewer higher-value gems.

Do not lose EXP value.

31. LEVEL SYSTEM

Track:

level currentEXP requiredEXP

Create configurable EXP progression.

Use data-driven or function-based scaling.

Keep function centralized.

When enough EXP exists for multiple levels:

Do not lose excess experience.

Queue multiple level-up selections if required.

32. UPGRADE SYSTEM

Upon Level Up:

Pause gameplay simulation.

Display 3 upgrade cards.

Valid cards include:

- acquire new weapon
- upgrade existing weapon
- acquire passive
- upgrade passive
- evolution

Do not show:

- already maxed upgrades
 - invalid evolution
 - new weapon if all weapon slots are full
 - new passive if all passive slots are full
-

33. UPGRADE CARD UI

Each card should show:

Icon Name Category Current level New level Description Main stat change

Example:

PULSE BLASTER

Lv2 → Lv3

Projectile Count 1 → 2

Cards must be large enough for mobile touch.

Pause combat while choosing.

34. RANDOM UPGRADE SELECTION

Create controlled random selection.

Support:

Seeded RNG.

Purpose:

- deterministic debugging
- reproducible tests

Prevent duplicate cards in one selection.

Future-ready for rarity weighting.

35. HUD

Portrait HUD must display:

Top left: Player level

Top center: Mission timer

Top right: Pause button

Below top: EXP bar

Player health: Clearly visible but compact.

Bottom region: Virtual joystick.

Show active weapon icons.

Do not obscure gameplay excessively.

36. GAME TIMER

Initial mission:

10:00.

Display:

MM:SS.

Stage timeline is driven by actual game elapsed time.

Paused time must NOT advance combat timer.

37. PAUSE SYSTEM

Pause button opens pause overlay.

Options:

Resume Restart Settings Quit to Menu

When paused:

Enemy movement stops. Weapons stop. Timer stops. Projectiles stop.

UI remains responsive.

38. MAIN MENU

Create professional simple main menu.

Display:

Game title Play Character Upgrades Settings

Initial version does not require elaborate artwork.

Use clean futuristic styling.

39. CHARACTER SELECT

Initial:

Aegis-01

Architecture must support multiple characters later.

Character data fields:

id name description baseStats startingWeapon unlockRequirement icon

40. STAGE SELECT

Initial stage:

Stage 01 — Abandoned Colony

Show:

stage name duration difficulty best time completion state

Prepare architecture for multiple stages.

41. GAME RESULT

Victory screen:

MISSION COMPLETE

Show:

Survival Time Enemies Defeated Boss Defeated Level Reached Damage Dealt Gold Earned

Defeat screen:

MISSION FAILED

Show same statistics.

Buttons:

Retry Main Menu

42. GOLD SYSTEM

Enemies may drop gold.

Gold is a persistent currency.

Save after mission.

Do not introduce real-money purchases.

43. META PROGRESSION

Create initial permanent upgrade system.

Possible upgrades:

Attack HP Movement Armor Pickup Range

Each permanent upgrade costs gold.

Use increasing costs.

Keep balancing values configurable.

44. SAVE SYSTEM

Implement persistent save.

Initial storage:

localStorage.

Save structure should contain:

version gold unlockedCharacters completedStages permanentUpgrades settings statistics

Example:

```
{ "version": 1, "gold": 1200, "completedStages": ["stage_01"], "settings": { "music": 0.7, "sfx": 0.8 } }
```

Validate loaded save data.

Handle corrupted data gracefully.

45. SAVE VERSIONING

Implement save version.

Example:

version: 1

Create architecture for future migration.

Do not destroy save data unnecessarily when schema changes.

46. AUDIO SYSTEM

Create centralized AudioManager.

Categories:

Master Music SFX

Initial sounds:

player hit enemy hit enemy death weapon fire EXP pickup level up button click boss warning victory defeat

If real audio files are unavailable:

The game must still run.

Use placeholders or disable missing audio safely.

47. VISUAL FEEDBACK

Combat must provide readable feedback.

Include:

Enemy hit flash Critical hit distinction Small explosion Player damage feedback Level-up effect Elite spawn warning Boss warning

Avoid excessive effects that damage mobile performance.

48. DAMAGE NUMBERS

Create pooled damage-number system.

Do not create one DOM element per damage event.

Damage number display may be throttled under high load.

Critical damage:

larger display.

Do not require special color choices unless visual design needs them.

49. COLLISION ARCHITECTURE

Avoid excessive physics bodies when possible.

Use Phaser Arcade Physics selectively.

For high-density enemies:

Consider custom distance checks or simplified collision logic.

Collision groups:

Player Enemies PlayerProjectiles EnemyProjectiles Pickups

Avoid unnecessary enemy-vs-enemy physics collision.

Enemies may use lightweight separation logic if required.

50. ENEMY CROWD MOVEMENT

Hundreds of enemies must not all occupy exactly the same point.

Implement lightweight crowd separation.

Do not use expensive full pathfinding.

Possible approach:

Steering vector = directionToPlayer

+ small separation force

Only inspect nearby entities through spatial grid.

51. GAME STATE MACHINE

Define explicit game states.

Examples:

BOOT MENU PLAYING LEVEL_UP PAUSED BOSS VICTORY DEFEAT

Do not rely on many unrelated boolean variables.

Use a clear state-management approach.

52. EVENT SYSTEM

Create centralized typed EventBus.

Example events:

PLAYER_DAMAGED PLAYER_DIED ENEMY_KILLED EXP_COLLECTED PLAYER_LEVEL_UP
UPGRADE_SELECTED ELITE_SPAWNED BOSS_SPAWNED BOSS_DEFEATED GAME_VICTORY GAME_DEFEAT

Avoid unnecessary direct cross-module dependencies.

53. DATA-DRIVEN ARCHITECTURE

Balance values should be stored in data/config modules.

Examples:

weapons.ts enemies.ts passives.ts stages.ts

Do not scatter magic numbers throughout gameplay scripts.

54. TYPE SAFETY

Use TypeScript properly.

Avoid:

any

unless absolutely necessary.

Create interfaces/types for:

WeaponDefinition EnemyDefinition StageDefinition UpgradeDefinition EvolutionRecipe
CharacterDefinition SaveData DamageInfo

55. MOBILE SAFE AREA

Support iPhone notches and browser UI.

Respect CSS environment variables:

`safe-area-inset-top` `safe-area-inset-bottom` `safe-area-inset-left` `safe-area-inset-right`

HUD and joystick must not be hidden behind unsafe areas.

56. RESPONSIVE SCALING

Portrait layout is primary.

Support different screen ratios.

Do not stretch sprites disproportionately.

Use Phaser Scale Manager.

Recommended:

`FIT_CENTER_BOTH`

while preserving logical coordinates.

57. PWA

Convert game into installable Progressive Web App.

Create:

`manifest.webmanifest`

Include:

`name` `short_name` `start_url` `display: standalone` `orientation` `icons`

Add basic service worker support.

Goal:

User can open game in Safari and use:

Add to Home Screen

Game should launch like a standalone Web App.

Do not rely on PWA features for essential gameplay.

58. OFFLINE SUPPORT

After assets are initially downloaded:

The game should be capable of loading offline where browser caching permits.

Cache:

HTML JavaScript bundles CSS essential assets

Do not cache unnecessarily large development files.

59. TOUCH BEHAVIOR

Disable unwanted:

Page scrolling Pull-to-refresh where possible Text selection Long-press selection Double-tap zoom interference

Do not break accessibility unnecessarily.

Game canvas should capture gameplay touches appropriately.

60. ORIENTATION HANDLING

If the user rotates to landscape:

Do not crash.

Either:

adapt layout

or

show a message recommending portrait orientation.

Initial priority:

portrait mode.

61. SETTINGS

Create settings panel.

Options:

Music volume SFX volume Damage numbers ON/OFF Screen shake ON/OFF Performance mode

Save preferences.

62. PERFORMANCE MODE

Add:

Quality: Auto High Balanced Performance

Performance mode may:

reduce particles reduce visual trails reduce damage numbers reduce decorative elements

It must not reduce critical gameplay information.

63. DEBUG OVERLAY

Development build only.

Toggle with:

F2

Display:

FPS Frame Time Enemies Projectiles Pickups Pool usage Player position Player HP Player level Game time

Mobile debug option may be hidden behind developer flag.

64. DEVELOPER CHEATS

Development mode only.

Implement keyboard debug commands:

F3: Add level.

F4: Add gold.

F5: Spawn elite.

F6: Spawn boss.

F7: Toggle invincibility.

F8: Kill all normal enemies.

Never expose developer controls prominently in production UI.

65. TESTING

Create tests for non-rendering systems where practical.

At minimum test:

EXP progression upgrade validation evolution requirements stat modifiers save/load damage calculations stage event timing

Use appropriate TypeScript test framework.

66. GAME BALANCE

Initial target:

Player should survive early game easily.

Mid game should require meaningful upgrade decisions.

Final two minutes should feel intense.

Boss should be challenging but fair.

Do not attempt perfect balance initially.

Expose important balance values in data files.

67. INITIAL NUMERICAL BALANCE

Create reasonable initial values yourself.

Do not ask the user to supply every stat.

Use engineering judgment.

Document assumptions.

Balance can be changed later.

68. MEMORY MANAGEMENT

Avoid unnecessary allocations in hot loops.

Avoid:

Array.filter every frame
Array.map every frame
temporary Vector2 creation every frame
frequent string concatenation
frequent object creation

Reuse arrays where practical.

Use preallocated working collections for critical queries.

69. UPDATE FREQUENCY

Not every system needs 60 Hz updates.

Examples:

Player movement: every frame.

Weapon cooldown: every frame or timer-based.

Spatial grid: scheduled or movement-aware.

Enemy targeting: 5–15 Hz may be sufficient.

Pickup merging: 1–2 Hz.

Performance analysis: 1 Hz.

Use sensible update rates.

70. RENDER OPTIMIZATION

Avoid complex real-time lighting.

Avoid expensive shader effects.

Avoid thousands of independent animated effects.

Use:

sprite batching texture atlases later simple particles pooled effects

Initial placeholder mode may use simple geometry.

71. PLACEHOLDER GRAPHICS

The game must run without finished artwork.

Create placeholder graphics programmatically.

Examples:

Player: circle/robot-like primitive.

Enemies: different geometric shapes.

Projectile: small energy circle.

EXP: diamond.

Boss: large distinctive shape.

All placeholder graphics should be isolated behind asset-loading architecture so they can be replaced later.

72. CAMERA

Camera follows player smoothly.

Avoid excessive smoothing latency.

Optional subtle screen shake:

Player damaged. Large explosion. Boss attack.

Allow screen shake to be disabled in Settings.

73. BATTLE STATISTICS

Track during a run:

Enemies killed Elite enemies killed Boss kills Damage dealt Damage received EXP collected Gold collected Highest damage hit Current DPS estimate Run duration

Use for ResultScene.

74. DPS TRACKER

Development/statistics feature.

Maintain rolling damage window.

Example:

last 5 seconds.

Calculate approximate DPS.

Do not calculate using expensive historical datasets.

75. UI DESIGN LANGUAGE

Style:

Engineering / futuristic interface.

Characteristics:

Dark high-tech background. Clear geometry. Strong information hierarchy. Minimal clutter. Readable mobile typography.

Avoid directly copying commercial game layouts.

76. ACCESSIBILITY

Ensure:

Text is readable. Important information is not communicated only through color. Touch buttons are sufficiently large. Critical warnings have visual pattern/shape.

77. PLAYER DEATH

When HP reaches zero:

Stop player control.

Trigger death animation/effect.

Pause combat shortly.

Open ResultScene.

Do not instantly reload page.

78. VICTORY CONDITION

Final boss defeated.

Trigger:

short victory effect.

Stop enemy spawning.

Clear unsafe projectiles if appropriate.

Show mission complete screen.

Save rewards.

79. BACKGROUND / TAB HANDLING

When browser loses focus:

Automatically pause game.

Do not allow player to be killed while browser tab is inactive.

When returning:

show paused state.

80. GAME SPEED

Use centralized time scaling.

Required for:

pause level-up future slow-motion effects

Do not arbitrarily modify timers independently.

81. ANTI-CHEAT

No serious anti-cheat is required for offline MVP.

However:

Separate runtime state from static data.

Do not over-engineer online security.

82. BACKEND

Do NOT build a backend for MVP.

Initial project must work completely client-side.

Prepare interfaces so backend/cloud save could be added later.

83. FUTURE BACKEND PREPARATION

Potential future systems:

Account login Cloud Save Leaderboards Daily Events Remote Configuration

Do not implement unless explicitly requested.

84. DEPLOYMENT

Prepare project for deployment to:

Vercel

and also compatible with:

Netlify Cloudflare Pages GitHub Pages

Provide:

`npm install`

`npm run dev`

`npm run build`

`npm run preview`

Ensure production build works.

85. README

Create complete README.md.

Include:

Project description Requirements Installation Development Build Deployment Controls Architecture
summary Folder structure Debug controls Known limitations

86. DEVELOPMENT WORKFLOW

Implement the project in phases.

Do NOT attempt to write the entire game as one giant file.

PHASE 1 — FOUNDATION

Create:

Vite TypeScript Phaser Game configuration Responsive canvas BootScene PreloadScene GameScene

Result:

A browser opens and displays the initial game world.

Verify:

npm run dev

works.

PHASE 2 — PLAYER

Create:

Player PlayerMovement Keyboard input Virtual joystick Camera follow

Verify:

Player moves smoothly on desktop and touch.

PHASE 3 — ENEMY

Create:

Basic enemy. Enemy pool. EnemyManager. SpawnDirector.

Verify:

Enemies continuously spawn and pursue player.

PHASE 4 — COMBAT

Create:

Pulse Blaster. Projectile pool. Targeting system. Damage system. Enemy death.

Verify:

Player automatically attacks and kills enemies.

PHASE 5 — EXPERIENCE

Create:

EXP drops. Pickup system. ExperienceSystem. LevelSystem.

Verify:

Killed enemies drop EXP and player levels up.

PHASE 6 — UPGRADE UI

Create:

Game pause on level-up. 3 upgrade cards. Upgrade selection. Resume gameplay.

Verify:

Upgrade choices affect gameplay.

PHASE 7 — ALL WEAPONS

Add all six initial weapons.

Verify each weapon independently.

PHASE 8 — PASSIVES

Add six passive modules.

Verify stat effects.

PHASE 9 — EVOLUTION

Add evolution system.

Verify correct requirements and upgraded behavior.

PHASE 10 — ENEMY VARIETY

Implement:

Crawler Scout Drone Heavy Drone Spitter Charger Swarm Unit

PHASE 11 — ELITE

Implement Elite Sentinel.

Verify timed spawning.

PHASE 12 — BOSS

Implement Guardian Prime.

Verify all three boss attacks.

Verify boss phases.

PHASE 13 — COMPLETE STAGE

Implement full 10-minute stage timeline.

Verify victory and defeat.

PHASE 14 — MAIN MENU

Create:

Main Menu Character Selection Stage Selection

PHASE 15 — META PROGRESSION

Create:

PHASE 16 — MOBILE OPTIMIZATION

Profile mobile performance.

Optimize:

enemy updates collisions projectiles EXP objects effects

PHASE 17 — PWA

Create:

manifest icons service worker offline caching standalone launch support

PHASE 18 — PRODUCTION BUILD

Run:

npm run build

Fix:

TypeScript errors build warnings missing assets runtime errors

Verify production bundle.

87. QUALITY GATES

After every major phase:

Run TypeScript compiler.

Run tests.

Run production build.

Do not proceed while known compilation errors exist.

Fix issues before continuing.

88. CODING RULES

Follow:

SOLID principles where appropriate.

Prefer composition.

Avoid unnecessary design-pattern complexity.

Keep systems understandable.

Methods should have clear responsibility.

Use descriptive variable names.

Avoid giant classes.

Avoid copy/paste logic.

89. ERROR HANDLING

The game must fail gracefully.

Examples:

Missing audio: continue without audio.

Invalid save: create safe default save.

Missing optional asset: use placeholder.

Invalid stage entry: fallback safely.

90. PERFORMANCE PRIORITY

When choosing between:

visually complex architecture

and

stable mobile performance

prefer stable mobile performance.

Gameplay clarity is more important than excessive effects.

91. FIRST PLAYABLE MILESTONE

The first important milestone must contain:

1 player

1 weapon

1 enemy type

enemy spawning

auto attack

enemy death

EXP collection

level up

3 upgrade choices

10-minute timer

basic victory state

This must be fully playable before unnecessary features are added.

92. MVP DEFINITION

Version 1.0 MVP must contain:

1 playable character

1 complete 10-minute stage

6 base weapons

6 passive modules

6 evolved weapons

6 standard enemy classes

1 elite enemy

1 final boss

EXP system

level-up system

upgrade selection

gold

basic permanent upgrades

save/load

main menu

pause

settings

result screen

mobile controls

PWA installation

production deployment

93. DO NOT IMPLEMENT YET

Do not initially build:

Multiplayer Gacha Advertising Real-money purchases Online accounts Chat PvP Complex inventory
Guilds Server-side leaderboards

Focus on excellent core gameplay.

94. REQUIRED TECHNICAL DOCUMENTATION

Create:

docs/ARCHITECTURE.md

Explain:

Scene architecture Combat architecture Enemy architecture Weapon architecture Event flow Pooling
Spatial grid Save architecture

Create:

docs/GAME_DESIGN.md

Explain:

Core loop Weapons Passives Enemies Boss Progression Stage timeline

Create:

docs/PERFORMANCE.md

Explain:

Performance targets Known bottlenecks Object pooling Spatial queries Mobile optimization

95. CODE COMMENTS

Comment:

non-obvious algorithms public system interfaces complex optimization logic

Do not fill every obvious line with comments.

96. COMPLETION REQUIREMENT

Do not claim a feature is complete merely because a placeholder class exists.

A feature is complete only when:

code exists it is connected to gameplay it runs it is testable it does not create known runtime errors

97. INITIAL RESPONSE FORMAT

Before implementing code, first return:

1. Architecture overview.
2. Technical decisions.
3. Project folder structure.
4. Core game-loop explanation.
5. Important performance strategy.
6. Implementation phases.
7. Initial balancing assumptions.
8. Risks and mitigation.

Then begin Phase 1.

Do not ask unnecessary questions.

Make reasonable engineering assumptions.

Clearly document them.

98. AFTER EACH PHASE

Report:

Completed Files created Files modified How the system works How to test Known limitations Next implementation step

Then continue implementation.

99. ACCEPTANCE TEST

The final project is accepted when:

A user can open the deployed URL on an iPhone.

The game loads without requiring an app installation.

The user can press Play.

The user controls the character using a touch joystick.

Enemies appear.

Weapons fire automatically.

Enemies can be defeated.

EXP can be collected.

Player can level up.

Three upgrade choices appear.

Player can select upgrades.

Weapons become stronger.

Elite enemies appear.

Boss appears at the end.

The game can end with victory or defeat.

Progress is saved.

Reloading the webpage retains persistent progress.

The page can be added to the iPhone Home Screen.

The game remains playable at acceptable mobile FPS.

100. START DEVELOPMENT

Begin now.

First inspect the existing repository.

If the repository is empty:

initialize the complete project.

If files already exist:

do not blindly overwrite working code.

Understand the existing architecture first.

Then create the architecture and implement Phase 1.

After Phase 1 is working, continue sequentially through the phases.

Always prioritize:

1. Working game
2. Correct architecture
3. Mobile performance
4. Maintainability
5. Visual polish

Do not prioritize visual polish over functional gameplay during the early phases.